UniFlow

Multi-Industry Operations Platform with an AI (MCP) Layer

Complete Technical Documentation

Backend · Frontend · MCP Server

Aviation & Retail

UniFlow — Project Documentation

> A multi-tenant, multi-industry operations platform with an AI (MCP) layer. This folder is the full technical documentation for the project — backend, frontend, and the MCP server.

---

1. What UniFlow is, in one paragraph

UniFlow is a SaaS-style "operations console" that reshapes itself depending on the industry of the company using it. A company signs up, picks an **industry** (today: **Aviation** or **Retail**), and gets a dashboard tailored to that business. An aviation company manages flights, cabin classes, and passenger bookings; a retail company (the brand owner) manages stores, products, and the inventory of each product in each store. On top of the normal web app, UniFlow exposes an **MCP server** — a bridge that lets an AI assistant (like ChatGPT) call the backend directly, so an end customer can search flights, book a ticket, or check whether a product is in stock, all through natural conversation.

The whole thing is **multi-tenant**: every company only ever sees its own data, and that isolation is enforced on the server from the logged-in user's identity, never from anything the browser sends.

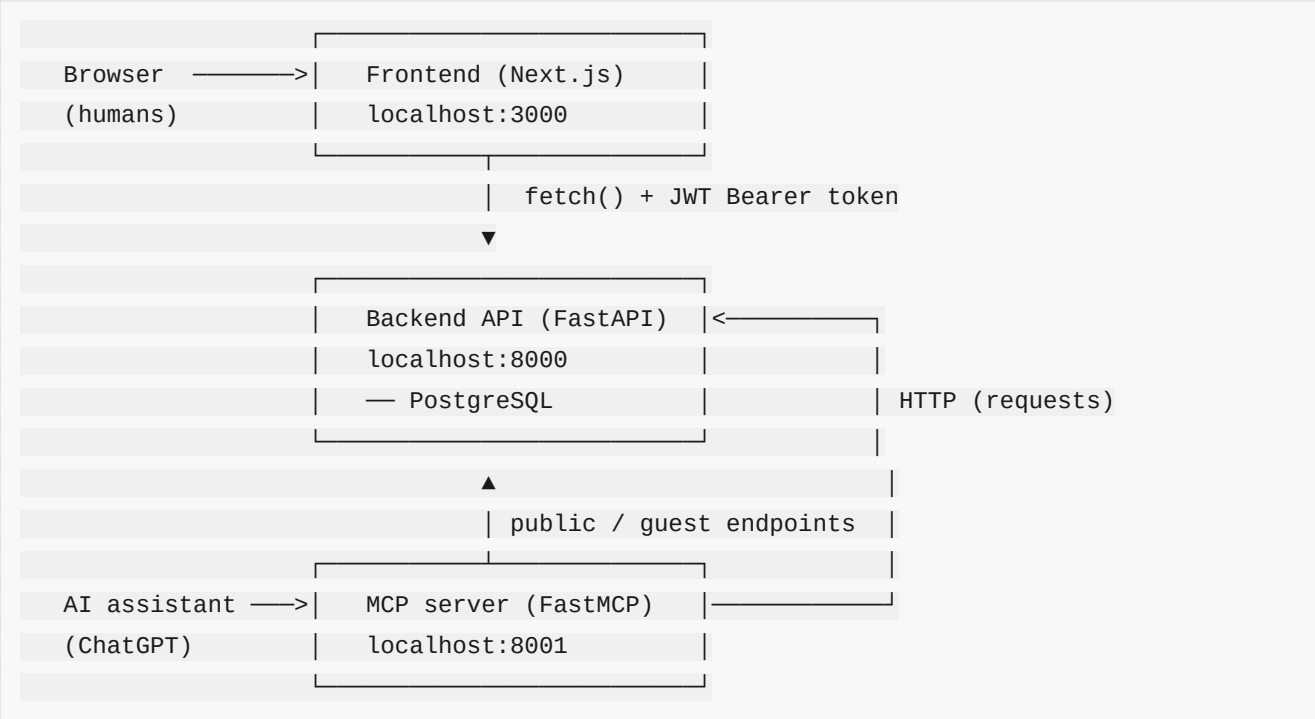
---

2. The three moving parts

UniFlow is really three cooperating services:

Service	Folder	Tech	Default port	Who talks to it
Backend	`backend/`	FastAPI +	`8000`	The frontend, the MCP server,

Service	Folder	Tech	Default port	Who talks to it
API		PostgreSQL		and (via public endpoints) AI tools
Frontend	`frontend/`	Next.js 16 (App Router) + React 19	`3000`	Human users in a browser
MCP server	`backend/app/mcp/server.py`	FastMCP	`8001`	An AI client (ChatGPT / any MCP-capable assistant)



The key idea: **the MCP server never touches the database directly.** It is a thin AI-facing wrapper that makes ordinary HTTP calls to the backend's public endpoints — exactly like the frontend does, just for a different kind of caller.

---

3. Technology stack

Backend - **FastAPI** — the web framework (routers, dependency injection, automatic OpenAPI docs at `/docs`). - **SQLAlchemy** (declarative ORM) over **PostgreSQL**. - **Alembic** — database migrations (schema versioning). - **python-jose** — JWT creation/verification (HS256). - **passlib + bcrypt** — password hashing. - **smtplib** (Gmail over SSL) — transactional email (verification, password reset, booking payment links). - **Pydantic** — request/response schemas and validation.

Frontend - **Next.js 16.2.6** (App Router) + **React 19** + **TypeScript**. - **Tailwind CSS v4** with a token-

based design system (CSS variables) and `next-themes` light/dark mode. - **lucide-react** icons, **@base-ui/react** primitives, **class-variance-authority** for component variants. - The initial UI scaffold was generated with Vercel **v0** (you'll still see `[v0]` console logs).

MCP - **FastMCP** — a Python framework for building Model Context Protocol servers, served here over **streamable-HTTP**.

---

4. How the pieces fit together (the mental model)

1. **A company registers.** `POST /auth/register-company` creates a `Company` row (with its chosen industry) and an **admin** `User`, then emails a verification link. Until the admin verifies, they can't log in.
2. **The admin logs in.** The backend returns a **JWT** (a signed token that encodes the user id, company id, and role). The frontend stores it in `localStorage` and attaches it as `Authorization: Bearer <token>` on every future request.
3. **The frontend decides which dashboard to show** based on two things read back from the server: the user's **role** (`admin` vs `employee`) and the company's **industry** (`aviation` vs `retail`). That 2×2 matrix picks one of four dashboards.
4. **Every authenticated request is tenant-scoped.** The backend reads `company_id` from the logged-in user (not from the request body) and filters all queries by it. One company can never read or write another's rows.
5. **Public/guest endpoints** exist alongside the tenant-scoped ones, for things a customer does without an account — searching flights, guest bookings, checking product availability. These are what the **MCP tools** call.
6. **Per-brand AI isolation.** Each company has an **API key**; its MCP server carries that key, so an AI app is bound to exactly one brand and can only ever read that brand's data — the end customer authenticates nothing, the *app* carries the identity. (Products also support **variant options** — colours, sizes, etc.)

---

5. The documentation map

Read in this order for a full understanding:

Doc	What's inside
README.md (this	The big picture, stack, how to run everything

Doc	What's inside
file)	
backend.md	Backend architecture, auth & security, multi-tenancy, every endpoint module, the email system
database.md	Every table, column, relationship, and constraint
frontend.md	The single-page controller, auth flow, the API layer, the industry-adaptive design, every component, the styling system
mcp.md	What MCP is, every tool, how it bridges to the backend, how to connect ChatGPT
api-reference.md	A compact lookup table of every HTTP endpoint
known-issues.md	Known bugs, limitations, and a future-work roadmap (useful for the PFE report)

---

6. Running the whole system locally

You need **three terminals** (backend, frontend, MCP) and a running **PostgreSQL** with a database named `pfe_backend_db`.

6.1 Backend (port 8000)

```
cd backend
source venv/bin/activate            # activate the virtualenv
alembic upgrade head                # apply all migrations (first run only)
uvicorn main:app --reload --port 8000
```

> [!] **Run uvicorn from *inside* `backend/`.** `main.py` lives there, and its imports are written as `app.…`. Launching from the project root gives `Could not import module "main"`.

Once it's up, open **http://localhost:8000/docs** for the interactive Swagger UI listing every endpoint.

The backend reads configuration from `backend/.env`:

Variable	Purpose
`DATABASE_URL`	PostgreSQL connection string

Variable	Purpose
`SECRET_KEY`	JWT signing secret
`EMAIL_USER` / `EMAIL_PASSWORD`	Gmail sender + app password (for outgoing mail)
`FRONTEND_URL`	Base URL used to build links inside emails (e.g. payment page)

6.2 Frontend (port 3000)

```
cd frontend
npm install        # first run only
npm run dev        # → http://localhost:3000
```

The frontend calls the backend at `process.env.NEXT_PUBLIC_API_URL`, defaulting to `http://localhost:8000`. Set that env var if your backend runs elsewhere. CORS on the backend currently allows exactly `http://localhost:3000`.

6.3 MCP server (port 8001)

```
cd backend
source venv/bin/activate
python app/mcp/server.py     # serves MCP over http on port 8001
```

The MCP server calls the backend at `BASE_URL = http://127.0.0.1:8000`, so **the backend must be running first**. See mcp.md for connecting an AI client.

---

7. A note on how this codebase was built

This is a final-year project (PFE). The aviation side (flights, bookings, employees, auth, email) was built first and is the more mature half. The **retail side** (stores with a responsible employee, products, per-store inventory, analytics) and the **retail MCP tools** were added later, deliberately mirroring the existing aviation patterns so the two halves feel like one product. Where the retail work required backend changes — a `manager_id` on stores, public read endpoints, mounting routers that were never wired up — those are documented in backend.md and known-issues.md.

Backend Documentation

The backend is a **FastAPI** application backed by **PostgreSQL**. It is the single source of truth for all data and business rules. Both the web frontend and the MCP server are just clients of this API.

---

1. Project structure

```
backend/
├── main.py                  # app entry point — creates FastAPI(), mounts routers, sets CORS
├── alembic/                 # database migrations
│   └── versions/            # one file per schema change
├── app/
│   ├── api/                 # the HTTP endpoints, one router per domain
│   │   ├── auth.py          #   /auth      — register, login, verify, password reset
│   │   ├── admin.py         #   /admin     — admin-only employee & flight & booking management
│   │   ├── account.py       #   /account   — self-service deactivate / delete
│   │   ├── company.py       #   /companies — read/update the company profile
│   │   ├── flight.py        #   /flights   — flights + public flight search
│   │   ├── booking.py       #   /bookings  — bookings + guest booking flow
│   │   ├── product.py       #   /products  — retail products + public search
│   │   ├── store.py         #   /stores    — retail stores + public search
│   │   └── inventory.py     #   /inventory — per-store stock + public availability
│   ├── models/              # SQLAlchemy ORM models (one class per table)
│   ├── schemas/             # Pydantic request/response models
│   ├── core/                # cross-cutting infrastructure
│   │   ├── database.py      #   engine, SessionLocal, get_db(), Base
│   │   ├── deps.py          #   get_current_user() — JWT validation dependency
│   │   ├── security.py      #   password hashing + JWT creation
│   │   └── email.py         #   Gmail SMTP send functions
│   └── mcp/
│       └── server.py        # the MCP server (documented separately in mcp.md)
```

The architecture is a clean layering: **router → schema (validation) → model (persistence)**, with `core/` providing the plumbing (DB sessions, auth, hashing, email) that every router leans on.

---

2. Application entry point (`main.py`)

`main.py` builds the FastAPI app and mounts **nine routers** — `auth`, `flight`, `booking`, `admin`, `account`, `company`, `product`, `store`, `inventory`. It also configures **CORS** to allow the frontend origin `http://localhost:3000` with any method/header and credentials.

The database schema is **not** auto-created at startup (`Base.metadata.create_all` is deliberately not called) — schema is owned entirely by Alembic migrations. There is no root (`/`) or health endpoint, so hitting `http://localhost:8000/` directly returns a `404` — that's expected; use `/docs`.

> **Historical note:** originally `main.py` imported the product/store/inventory/admin/account routers but only *mounted* auth, flight, and booking (and mounted `auth` twice). That meant the entire retail API and the admin/account endpoints returned `404` even though the code existed. This was fixed by mounting all nine routers and removing the duplicate. It's the kind of bug that hides in plain sight because each file looks correct in isolation.

---

3. Authentication & security

3.1 How a token is created

When a user registers or logs in, the backend calls `create_access_token()` (in `core/security.py`). This builds a **JWT** signed with `SECRET_KEY` using the **HS256** algorithm. The token payload contains:

- `sub` — the user's id (as a string)
- `company_id` — the tenant id
- `role` — `"admin"` or `"employee"`
- `exp` — an expiry timestamp

The expiry is `ACCESS_TOKEN_EXPIRE_MINUTES = 30 * 700 = 21000` minutes — about **14.6 days**. (That's a long-lived token; see known-issues.md.)

3.2 How a token is validated

Every protected endpoint depends on `get_current_user()` (in `core/deps.py`). It:

1. Reads the `Authorization: Bearer <token>` header (via FastAPI's `HTTPBearer`).
2. Decodes and verifies the JWT (`jwt.decode` also enforces `exp`; any problem → `401`).
3. Pulls `sub` (the user id) out of the payload.

4. **Loads the fresh `User` row from the database** and returns it.

The important design choice here: `get_current_user` always reads the **live** user from the DB rather than trusting the `company_id`/`role` baked into the token. So if an admin changes a user's role, the change takes effect on the user's next request — the token doesn't need to be reissued.

3.3 The admin guard

Admin-only routes depend on `require_admin()` (in `api/admin.py`), which wraps `get_current_user` and raises `403 "Admins only"` if `role != "admin"`. A couple of endpoints (`company.py`'s update, `auth.py`'s employee registration) re-implement the same check inline instead of reusing `require_admin`.

3.4 Passwords

Passwords are hashed with **bcrypt** via passlib's `CryptContext`. `hash_password()` and `verify_password()` are the only two functions the rest of the app uses — plaintext passwords never touch the database.

3.5 Email verification & password reset tokens

Both flows share one table, `email_verification_tokens`, and one mechanism:

- A random token is generated with `secrets.token_urlsafe(32)`.
- It's stored with an `expires_at` and a `used` boolean.
- **Email verification** tokens live **24 hours**; **password reset** tokens live **1 hour**.
- Consuming a token checks it exists, isn't `used`, and isn't expired, then flips `used = True`.

---

4. Multi-tenancy — how companies stay isolated

This is the single most important concept in the backend. **The tenant is the `Company`.** Almost every table (`users`, `flights`, `bookings`, `products`, `stores`, `inventory`, …) carries a `company_id` foreign key.

The rule the whole backend follows:

> The tenant id always comes from the **authenticated user** (`current_user.company_id`), never from the request body.

That's why request schemas like `CreateStore`, `ProductCreate`, and `CreateInventory` deliberately **omit** `company_id` — there are comments in the code saying "provided by the token." A malicious client can't set `company_id` to someone else's company because the server ignores any such field.

Concretely:

- **Reads** filter by `company_id == current_user.company_id`. A store, product, flight, or booking from another company simply isn't in the result set.
- **Writes** stamp `company_id = current_user.company_id` onto new rows.
- **Cross-entity checks** enforce tenancy too. For example, when you assign a store manager, `_validate_manager` confirms that user belongs to your company; when you add inventory, the backend confirms both the product and the store are yours.
- **Cross-tenant access returns `404`**, not `403` — the backend treats "not yours" as "doesn't exist," which avoids leaking whether a given id exists at all.
- **Uniqueness is per-tenant**: product SKUs are unique per company, inventory is unique per (company, product, store).

The exceptions are the **public/guest endpoints** (see §6), which intentionally bypass tenancy so external customers and AI tools can search across companies.

---

5. The endpoint modules

Below, each module is described in plain English. For an exhaustive parameter-level table, see api-reference.md.

5.1 `auth.py` — identity (`/auth`)

The front door of the system.

- **`POST /auth/register-company`** — the sign-up entry point. Creates the `Company` (with its chosen industry) and the first `User` as an **admin**, hashes the password, issues a 24-hour email-verification token, and sends the verification email. It returns a JWT immediately, but the user still can't log in until they verify.
- **`POST /auth/Login`** *(note the capital "L")* — looks up the user by email, verifies the password, and **blocks unverified users** with a `403`. On success returns a JWT plus `company_id`, `role`, and `name`.
- **`GET /auth/verify-email?token=…`** — consumes a verification token and flips the user's `is_verified` to true.
- **`POST /auth/resend-verification`** — issues a fresh verification token (returns a deliberately neutral message so it can't be used to probe which emails exist).
- **`POST /auth/forget-password`** / **`POST /auth/reset-password`** — the reset pair: request a 1-hour token by email, then submit that token with a new password.

- **`PUT /auth/change-password`** — for a logged-in user changing their own password (needs the current password).
- **`POST /auth/register-employee`** — an **admin** creates a staff member inside their own company. The new user is created already verified and active. (In retail, these are the people who become store managers.)
- **`GET /auth/me`** — returns the logged-in user's profile **joined with their company**, so it includes `company_name` and `industry`. The frontend relies on this to know which dashboard to render.

> **Retail-relevant fix:** `/auth/me` originally didn't return `industry` or `company_name`. The frontend read `me.industry` as `undefined` and fell back to `"aviation"` — so a retail company would log in and see the aviation dashboard. Adding those two fields fixed it. See known-issues.md.

5.2 `admin.py` — administration (`/admin`, admin-only)

Everything here is gated by `require_admin`.

- **Employee management:** list all employees in the company, update an employee (name/role/active), disable an employee, and reset an employee's password (an admin override that doesn't need the old password). Guards prevent an admin from demoting or disabling themselves into a lockout.
- **Flight management:** update any field on a flight (including status), and delete a flight — with a safety check that refuses deletion while any booking is still `reserved`.
- **Booking detail:** fetch one booking enriched with its flight and class information.

5.3 `account.py` — self-service (`/account`)

Lets a user manage their **own** account (password required to confirm):

- **Deactivate** — set yourself inactive. Blocked if you're the company's only active admin (prevents orphaning the company).
- **Delete** — an employee deletes only their own row; an admin with co-admins does the same; but the **last remaining admin** deleting their account triggers a full company wipe (bookings → flight classes → flights → users → company).

5.4 `company.py` — company profile (`/companies`)

- **`GET /companies/me`** — the caller's company.
- **`PUT /companies/me`** — admin-only partial update of name / industry / email.
- **`GET /companies/me/api-key`** — admin-only; returns the brand's **MCP API key** (lazily minted if missing). The admin pastes this into their MCP server (`COMPANY_API_KEY`).

- **`POST /companies/me/api-key/rotate`** — admin-only; invalidates the old key and issues a new one.

5.5 `flight.py` — flights (`/flights`) *(aviation)*

- **`POST /flights/`** — create a flight together with its cabin classes (economy/business/first), seeding `available_seats = total_seats` for each.
- **`GET /flights/`** — list your company's flights, optionally filtered by origin/destination.
- **`GET /flights/{id}`** — one flight (tenant-scoped).
- **Public:** `GET /flights/public/search` (search by origin/destination/company name), `GET /flights/{id}/available` (seats by class), `GET /flights/{id}/status`. These have no auth and no tenant filter — they're for customers and the MCP tools.

5.6 `booking.py` — bookings (`/bookings`) *(aviation)*

Two audiences share this module:

- **Authenticated (staff) side:** create a booking, list bookings, cancel a booking (which returns a seat to inventory).
- **Guest side (`/bookings/guest/*`, public):** the customer-facing flow that the MCP tools and the payment page use — create a guest booking (decrements a seat and emails a payment link), look up a booking by id + passenger email, change the passenger email, pay (a **simulated** payment that just flips `payment_status` to `paid`), and cancel.

5.7 `product.py` — products (`/products`) *(retail)*

- **`POST /products/`** — create a product; rejects a duplicate SKU within the company. Accepts an optional **`attributes`** list (variant options like colour/size).
- **`GET /products/`** — list your company's products (including their attributes).
- **Public:** `GET /products/public/search?query=&company_name=` (matches name **or** SKU) and `GET /products/public/{id}`. Used by the retail MCP tools.

5.8 `store.py` — stores (`/stores`) *(retail)*

This module was extended to support the "each store has a responsible employee" requirement.

- **`POST /stores/`** — create a store with a name, location, and an optional `manager_id`. The manager, if given, must be a user in your company (`_validate_manager`).
- **`GET /stores/`** — list stores, each enriched with the manager's name (`_serialize` looks it up from the users table).

- **`PUT /stores/{id}`** — partial update. It uses Pydantic's `model_fields_set` so that sending `manager_id: null` explicitly **un-assigns** a manager, while omitting the field leaves it untouched.
- **`DELETE /stores/{id}`** — deletes the store, clearing its inventory rows first to respect foreign keys.
- **Public:** `GET /stores/public/search?location=&company_name=`. Powers the "where are your shops?" MCP tool.

5.9 `inventory.py` — per-store stock (`/inventory`) *(retail)*

This ties products and stores together: an inventory row says "store X holds Y units of product Z."

- **`POST /inventory/`** — assign stock. If a row already exists for that (company, store, product), it **adds** to the existing quantity; otherwise it creates a new row. Both the product and the store must belong to your company.
- **`GET /inventory/`** — list all inventory rows for your company (raw ids; the frontend joins names client-side).
- **`PUT /inventory/{id}`** — **set** (overwrite) the quantity — this is what a store employee uses to correct stock.
- **`DELETE /inventory/{id}`** — remove a stock record.
- **Public:** `GET /inventory/public/availability?product_name=&location=&company_name=` — joins inventory → product → store and returns only rows with `quantity > 0`, in a flat human-readable shape (product name, store name, store location, quantity). This one endpoint powers **two** MCP tools: "is X in stock near me?" and "which store has X?".

---

6. Public vs. authenticated endpoints

It's worth calling out the two "tiers" of the API explicitly, because it's central to how the MCP layer works:

- **Authenticated, tenant-scoped** — the vast majority. Require a Bearer token, filter by the caller's company. Used by the frontend.
- **Public / guest** — no token, no tenant filter, optional `company_name` filter. Used by external customers and the MCP tools. These are: all `/flights/public/*` and `/flights/{id}/available|status`, all `/bookings/guest/*`, `/products/public/*`, `/stores/public/search`, and `/inventory/public/availability`.

The public tier is what makes the AI integration possible — an AI assistant has no login, so it can only reach the endpoints that don't require one.

6.1 Per-brand scoping with an API key (`X-API-Key`)

The public discovery endpoints (`/flights/public/search`, `/products/public/*`, `/stores/public/search`, `/inventory/public/availability`) accept an optional **`X-API-Key`** header. It's resolved by the `company_id_from_api_key` dependency (`core/deps.py`):

- **Key present** → the request is scoped to *that* company only; `company_name` is ignored, and another brand's data is invisible (a cross-brand product-by-id returns `404`). An invalid key → `401`.
- **Key absent** → the endpoint falls back to the optional `company_name` filter (backward-compatible).

This is how a customer-facing AI app is bound to **one brand**: the *company* (the app instance) carries the identity via its key, while the end customer authenticates nothing. Each company has a unique `api_key` (generated at registration, retrievable/rotatable via `/companies/me/api-key`). The MCP server reads it from the `COMPANY_API_KEY` environment variable — see mcp.md.

---

7. The email system (`core/email.py`)

Email is sent through **Gmail over SMTP-SSL** (`smtp.gmail.com:465`) using stdlib `smtplib`, authenticated with `EMAIL_USER` / `EMAIL_PASSWORD`. Every send function returns `{"success": bool, "message": str}` and **never raises** — if mail fails, the API request still succeeds and the failure is just logged. There are three flows:

Function	Triggered by	Sends
`send_verification_email`	register-company, resend-verification	an email-verification link
`password_resert_email`	forget-password	a password-reset link
`email_payment_send`	guest booking create + email change	a payment link (`{FRONTEND_URL}/pay/booking/{id}`)

---

8. Database migrations (Alembic)

Schema changes live in `backend/alembic/versions/`, each file chaining to the previous via `down_revision`. To apply everything to a fresh database:

```
alembic upgrade head
```

> **Gotcha we hit:** on this project's dev database, the `alembic_version` table existed but was **empty** (the schema had been created outside Alembic tracking). Running `alembic upgrade head` then tried to run *every* migration from scratch and collided with already-existing tables. The fix was to **stamp** the database at the last-known revision (`alembic stamp <rev>`) and then upgrade, so Alembic only applied the genuinely new migration (`add_manager_id_to_stores`). If you ever see "relation already exists" on upgrade, that's the cause.

---

9. Where the "unused" models come from

Five models are defined and imported but no endpoint reads or writes them: `api_integrations`, `endpoints`, `mcp_tool_permissions`, `rate_limits`, and `audit_logs`. They're **forward-looking scaffolding** for a richer MCP/integration layer — the ability to register external APIs, gate which MCP tools a company may use, rate-limit, and audit actions. They're documented in database.md but there's no live behaviour behind them yet.

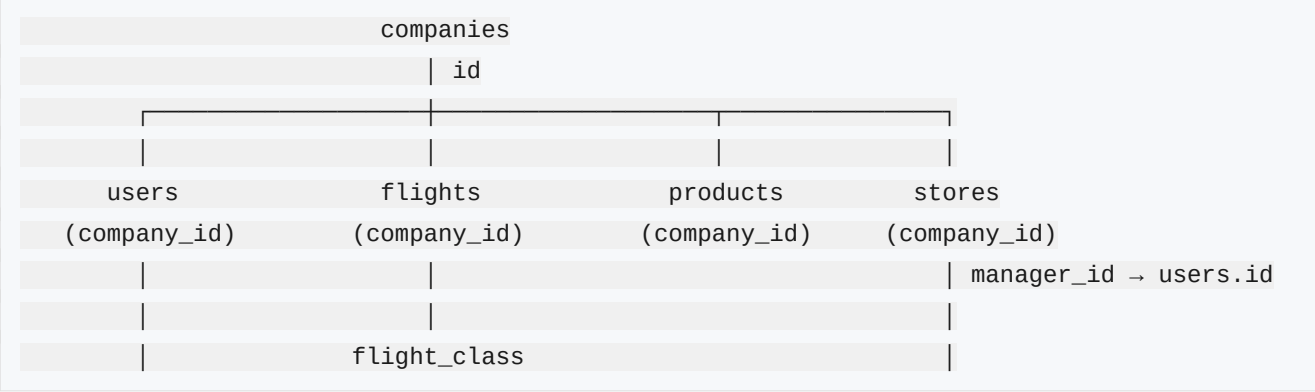
For a full list of the sharper edges (the `email_status` tuple bug, the `GET /bookings/` single-row limitation, the `is_active` login check, etc.), see **known-issues.md** — they're gathered there so this document can stay focused on how the system is *meant* to work.

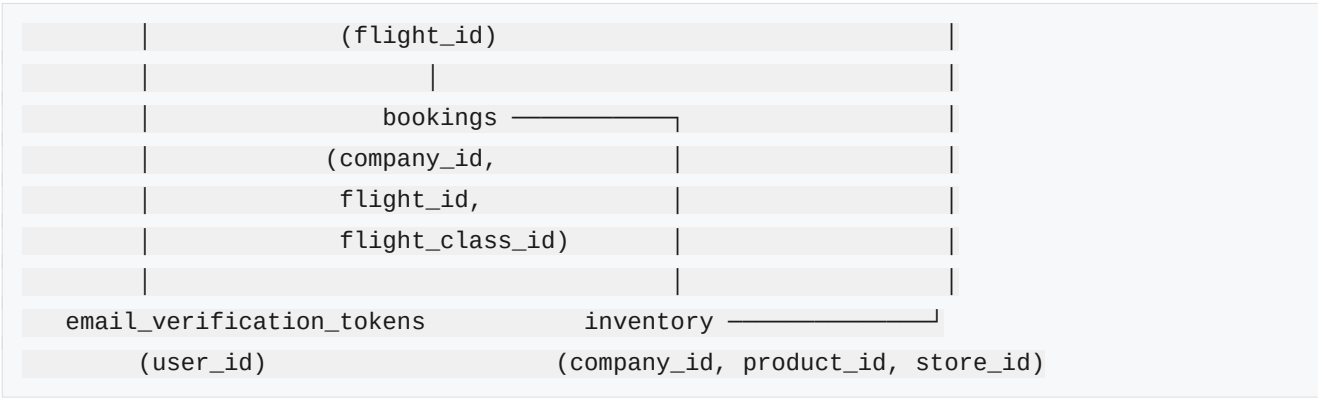
Data Model

The database is **PostgreSQL**, modelled with SQLAlchemy's declarative ORM (`app/models/`). Every domain table hangs off the `companies` table by a `company_id` foreign key — that FK is the backbone of multi-tenancy.

---

1. The big picture





- **`companies`** is the tenant/root.
- **`users`** are the people (admins and employees) belonging to a company.
- Aviation branch: **`flights`** → **`flight_class`** → **`bookings`**.
- Retail branch: **`products`** and **`stores`**, joined by **`inventory`** (which product, in which store, how many). A store optionally points to a user via **`manager_id`** (its responsible employee).
- **`email_verification_tokens`** backs both email verification and password reset.

---

2. Core tables

`companies` — the tenant

Column	Type	Notes
`id`	Integer, PK	
`name`	String	NOT NULL
`industry`	Enum(`'retail'`,`'aviation'`)	NOT NULL, default `'aviation'`
`email`	String	NOT NULL, **unique**
`status`	String	NOT NULL, default `'active'`
`api_key`	String	**unique**, indexed, nullable — the brand's MCP / AI-app key (see mcp.md)
`created_at`	DateTime(tz)	server default `now()`

The `api_key` binds an AI app / MCP server to exactly one brand: a request carrying it (via the `X-API-Key`

header) is scoped to this company only. The `industry` enum only has two values (`retail`, `aviation`) — this is what drives which dashboard a company sees. There's no `general` value even though the frontend's design config mentions one.

`users` — people

Column	Type	Notes
`id`	Integer, PK	
`company_id`	Integer, FK → companies.id	NOT NULL
`name`	String	NOT NULL
`email`	String	NOT NULL, **unique globally**
`password_hash`	String	NOT NULL (bcrypt)
`role`	String	NOT NULL, default `"Admin"` (see note)
`is_active`	Boolean	default True
`is_verified`	Boolean	default False
`created_at` / `updated_at`	DateTime(tz)	`updated_at` auto-updates

Two things worth knowing: - **Email is globally unique**, not per-company — the same email address can't be reused across two different companies. - The **column default for `role` is `"Admin"` (capitalized)**, but every code path explicitly sets `"admin"` / `"employee"` (lowercase) and every check compares against lowercase. The capitalized default is never actually used — but if it ever were, admin checks would silently fail. See known-issues.md.

---

3. Aviation tables

`flights`

Column	Type	Notes
`id`	Integer, PK	

Column	Type	Notes
`company_id`	Integer, FK → companies.id	NOT NULL
`flight_number`	String	NOT NULL
`origin` / `destination`	String	NOT NULL
`departure_time` / `arrival_time`	DateTime	NOT NULL
`status`	String	NOT NULL, default `"scheduled"`

Relationship: `Flight.classes` → the flight's cabin classes (backref to `flights`).

`flight_class`

Column	Type	Notes
`id`	Integer, PK	
`flight_id`	Integer, FK → flights.id	NOT NULL
`flight_class`	Enum(`economy`,`business`,`first`)	NOT NULL
`price`	Integer	NOT NULL
`total_seats`	Integer	NOT NULL
`available_seats`	Integer	NOT NULL

Each flight has 1–3 classes. `available_seats` starts equal to `total_seats` and is decremented on booking, restored on cancellation.

`bookings`

Column	Type	Notes
`id`	Integer, PK	

Column	Type	Notes
`company_id`	Integer, FK → companies.id	NOT NULL
`flight_id`	Integer, FK → flights.id	NOT NULL
`flight_class_id`	Integer, FK → flight_class.id	NOT NULL
`passenger_name` / `passenger_email`	String	NOT NULL
`status`	String	NOT NULL, default `"reserved"` (→ `"cancelled"`)
`payment_status`	String	NOT NULL, default `"unpaid"` (→ `"paid"`)
`payment_link`	String	nullable
`email_status`	String	NOT NULL, default `"not_sent"`
`email_error`	String	nullable
`created_at`	DateTime(tz)	server default now

A booking records both the reservation state and the (simulated) payment/email state. Note `company_id` is stored even on **guest** bookings — it's inherited from the flight's company so the owning company can see bookings made against its flights.

---

4. Retail tables

`products`

Column	Type	Notes
`id`	Integer, PK	
`company_id`	Integer, FK → companies.id	NOT NULL

Column	Type	Notes
`sku`	String	NOT NULL
`name`	String	NOT NULL
`description`	String	nullable
`price`	Float	NOT NULL
`attributes`	JSON	nullable — variant options, e.g. `[{"name":"Color","values":["Red","Blue"]}, {"name":"Size","values":["40","41"]}]`
`created_at`	DateTime(tz)	

`attributes` records the **available options** a product comes in (colours, sizes, or any custom attribute). It captures the options, not separate stock per variant. Unique constraint **`unique_product_sku_per_company`** on (`company_id`, `sku`) — a SKU is unique within a company but two different companies can use the same SKU.

`stores`

Column	Type	Notes
`id`	Integer, PK	
`company_id`	Integer, FK → companies.id	NOT NULL
`name`	String	NOT NULL
`location`	String	NOT NULL
`manager_id`	Integer, FK → users.id	**nullable**
`created_at`	DateTime(tz)	

`manager_id` is the "responsible employee" — a nullable link to a user in the same company. It was **added specifically for the retail feature** (model change + the `add_manager_id_to_stores` Alembic migration, with `ON DELETE SET NULL` so deleting a user doesn't break their stores). A store with no manager is "unassigned."

> There was an intended unique constraint on (`company_id`, `name`), but a typo in the model (`__table_arg__` instead of `__table_args__`, and not a tuple) means **it is silently not created**. Two stores in the same company can currently share a name. See known-issues.md.

`inventory` — the product×store join

Column	Type	Notes
`id`	Integer, PK	
`company_id`	Integer, FK → companies.id	NOT NULL
`product_id`	Integer, FK → products.id	NOT NULL
`store_id`	Integer, FK → stores.id	NOT NULL
`quantity`	Integer	NOT NULL, default 0
`created_at` / `updated_at`	DateTime(tz)	`updated_at` auto-updates

Unique constraint **`unique_inventory_per_store`** on (`company_id`, `product_id`, `store_id`) — there is exactly one stock row per product per store, which is why "add stock" increments rather than inserting duplicates.

This table is the heart of the retail model: it answers "how many of product P are in store S?" and, aggregated, powers the analytics (total units, inventory value = Σ quantity × price, low-stock alerts) and the public availability search.

---

5. Auth / token table

`email_verification_tokens`

Column	Type	Notes
`id`	Integer, PK	
`user_id`	Integer, FK → users.id	NOT NULL
`token`	String	NOT NULL, **unique**,

Column	Type	Notes
		indexed
`expires_at`	DateTime(tz)	NOT NULL
`used`	Boolean	default False
`created_at`	DateTime(tz)	server default now

One table, two jobs: email-verification tokens (24 h) and password-reset tokens (1 h). The row doesn't record which kind it is — the flow that consumes it defines the meaning.

---

6. Scaffolding tables (defined, not yet used)

These five are declared as models (so Alembic manages them) but no endpoint reads or writes them. They sketch out a future MCP/integration platform:

Table	Intended purpose
`api_integrations`	Register an external API a company wants the platform to call (base URL, auth type, an `encrypted_secret`).
`endpoints`	Individual endpoints belonging to a registered integration (path, method, type).
`mcp_tool_permissions`	Per-company on/off switch for individual MCP tools (`tool_name`, `is_enabled`).
`rate_limits`	A rolling request counter per company per resource type (windowed rate limiting).
`audit_logs`	An audit trail (`action`, `entity_type`, `entity_id`, actor `user_id`, JSON metadata).

They're a good indication of where the project intends to go — a governed, auditable, rate-limited AI-integration layer — but today they're inert. **Audit logging is therefore not yet implemented** anywhere.

---

7. A note on the duplicate `User`/`Users` models

There are two model files that both map to the `users` table: `models/user.py` (`User`) and `models/users.py` (`Users`). **Only `User` is used** by the application; `Users` is legacy/dead code (it isn't

imported in `models/__init__.py`, lacks `is_verified`/`updated_at`, and marks `password_hash` unique). Mapping two classes to one table is a latent hazard and one of them should be deleted — flagged in known-issues.md.

Frontend Documentation

The frontend is a **Next.js 16** (App Router) + **React 19** + **TypeScript** application, styled with **Tailwind CSS v4** and a token-based design system. Interestingly, almost the entire app runs as a **single client-side page** driven by a state machine — Next's routing is only used for a handful of auxiliary pages (email verification, password reset, guest payment).

---

1. The app shell (`app/layout.tsx`)

The root layout is a server component that:

- Loads three Google fonts as CSS variables: **Geist** (sans, `--font-geist-sans`), **Geist Mono** (`--font-geist-mono`), and **Instrument Serif** (`--font-instrument-serif`, used for display headings).
- Sets the page metadata (`title`: "UniFlow — Multi-Industry Workflow Infrastructure").
- Wraps everything in **`next-themes`** (`ThemeProvider attribute="class" defaultTheme="system"`) so the app supports light/dark following the OS, toggled by adding/removing a `.dark` class on `<html>`.
- Mounts Vercel Analytics only in production.

---

2. The single-page controller (`app/page.tsx`)

This is the brain of the app. Rather than route between pages, it keeps a **`view` state machine** and swaps whole screens in and out:

```
landing → auth → loading → dashboard
                ↘ forgot-password
```

State it tracks: `view` (which screen), `authMode` (signup/login), `company`, `industry`, `userRole`, a `refreshKey`, and a `restoring` flag.

Session restore: on mount, if there's an `authToken` in `localStorage`, it reads the saved `role`, `companyName`, and `industry` and jumps straight to the dashboard — so a returning user doesn't see the landing page again.

The dashboard selection matrix is the important part. Once logged in, the app chooses one of **four** dashboards from the combination of **role** and **industry**:

	Aviation	Retail
Admin	`AdminDashboard`	`RetailAdminDashboard`
Employee	`EmployeeDashboard`	`RetailEmployeeDashboard`

```
if (userRole === "admin")
  AdminView = industry === "retail" ? RetailAdminDashboard : AdminDashboard
if (userRole === "employee")
  EmployeeView = industry === "retail" ? RetailEmployeeDashboard : EmployeeDashboard
```

Dashboards are mounted with `key={refreshKey}`; the sidebar's "Refresh" button increments that key to force a clean remount (a simple way to re-fetch everything).

> There's also a **fallback** `Dashboard` component (the config-driven one) that only renders when `userRole` is `null`. Because login always stores a role, this branch is effectively unreachable in normal use — see §7.

---

3. The authentication flow (frontend side)

Signup & login (`components/auth-screen.tsx`)

A two-column screen (branding on the left, form on the right). The industry dropdown offers **Aviation** and **Retail** only ("General" was intentionally removed because the backend enum doesn't support it and there's no General dashboard).

- **Signup** calls `signupUser(...)` (→ `POST /auth/register-company`). It does **not** log the user in. Instead it shows a "**Verify your email to continue**" screen with a resend button — the user must click the link in their inbox, then come back and log in.
- **Login** calls `loginUser(...)` (→ `POST /auth/Login`), then hands the real `company_name` and `industry` (fetched from `/auth/me`) up to `page.tsx`, which routes to the correct dashboard.
- Raw backend errors are translated into friendly copy ("Incorrect email or password", "Please

verify your email before signing in", etc.).

What's stored in `localStorage`

After a successful login, `loginUser` persists eight keys (all reads/writes are centralized in `lib/api.ts`):

`authToken`, `userRole`, `companyId`, `userName`, `companyName`, `industry`, `userId`, `userEmail`.

`userId` in particular is used by the **retail employee dashboard** to figure out which store(s) the logged-in employee manages (`store.manager_id === getUserId()`). `clearAuth()` removes all eight on logout.

Password reset & email verification (auxiliary real routes)

These are the only parts that use actual Next.js routing, because they're links people click from emails:

- **`/verify-emai?token=…`** — the email-verification landing page (the folder name is misspelled "verify-emai"; the backend links here to match). Calls `verifyEmail(token)`, shows success/expired/invalid states, offers a resend.
- **`/reset-password?token=…`** and **`/verify-password?token=…`** — both render the reset-password screen (`resetPassword(token, newPassword)`).
- **`/pay/booking/[id]?email=…`** — the **public guest payment page**. No login. Looks up a guest booking by id + email, shows a summary, and "pays" (`payGuestBooking`). This is the page the booking-confirmation email links to.
- **`/forgot-password`** flow (in-app view) collects an email and calls `requestPasswordReset`.

---

4. The API layer (`lib/api.ts`)

Every network call goes through one helper, `apiCall<T>(endpoint, options)`:

- **Base URL** comes from `NEXT_PUBLIC_API_URL` (default `http://localhost:8000`).
- **Token injection:** it reads `authToken` from `localStorage` and, if present, sets `Authorization: Bearer <token>`. So authentication is automatic — individual calls don't worry about it.
- **Error normalization:** on a non-OK response it digs the message out of FastAPI's various error shapes (`detail` as a string, `detail[0].msg` for validation errors, or `message`) and throws an `Error` with a `.status` property so callers can branch on the HTTP code.

A few **public/guest** functions deliberately bypass `apiCall` and use a raw `fetch` with no auth header — `searchFlights`, `bookFlight`, `getGuestBookingDetails`, `payGuestBooking` — because those endpoints are meant to work without a login.

The exported functions are grouped by domain. A selection:

Domain	Functions
Auth	`signupUser`, `loginUser`, `logoutUser`, `getCurrentUser`, `verifyEmail`, `resendVerification`, `requestPasswordReset`, `resetPassword`, `changePassword`
Flights	`createFlight`, `getFlights`, `getFlightById`, `getAvailableSeats`, `searchFlights`, `updateFlight`, `deleteFlight`
Bookings	`getBookings`, `getBookingDetails`, `bookFlight`, `getGuestBookingDetails`, `payGuestBooking`
Employees	`getEmployees`, `createEmployee`, `updateEmployee`, `disableEmployee`, `enableEmployee`, `changeEmployeePassword`
Account	`deactivateAccount`, `deleteAccount`
Retail — Stores	`getStores`, `createStore`, `updateStore`, `deleteStore`
Retail — Products	`getProducts`, `createProduct`
Retail — Inventory	`getInventory`, `addInventory`, `setInventoryQuantity`, `deleteInventory`
Helpers	`isAuthenticated`, `getAuthToken`, `clearAuth`, `getUserId`, `getUserRole`, `getUserName`, `getUserEmail`, `getCompanyName`, `getIndustry`, `isAdmin`, `isEmployee`

The exported TypeScript interfaces (`Store`, `Product`, `InventoryItem`, `Flight`, `Booking`, `Employee`, `CurrentUser`, `TokenResponse`, …) are the contract between the components and the backend.

---

5. The industry-adaptive design system (`lib/industries.ts`)

This file defines an `IndustryConfig` for each industry — its name, tagline, icon, navigation items, stat cards, a staff directory, and an operations table. There are three configs: **aviation**, **retail**, and **general**.

Important caveat: everything in this file is **hardcoded mock data** — the "42 active flights", "$1.24M revenue", the sample staff rows, the fake orders. It feeds the **config-driven** `Dashboard` component (§7), which is the *demo* dashboard, not the real one. The real, data-backed dashboards (§6) don't use this file at all. It's essentially a designed placeholder / marketing preview of what each industry mode looks like.

---

6. The real dashboards (role-based, data-backed)

These four are what a logged-in user actually uses. They share a common shape: a collapsible sidebar with a brand header, a list of tab buttons, and Refresh / Sign-out at the bottom; the main area renders one **view component** per tab.

Aviation — Admin (`admin-dashboard.tsx`)

Tabs: **Flights · Employees · Bookings · Profile**. - `AdminFlightsView` — the flagship CRUD screen: create flights (with multiple cabin classes), search, click a row to open a modal and edit status, delete (with a guard). Colored status badges. - `AdminEmployeesView` — full employee management (create, edit, enable/disable, reset password). - `AdminBookingsView` — read-only bookings with stat tiles (total/reserved/paid/revenue) and status/payment filters. - `ProfileView` — account info + change password.

Aviation — Employee (`employee-dashboard.tsx`)

Tabs: **Flights · Bookings · Settings**. It **reuses** `AdminFlightsView` and `AdminBookingsView` (so employees currently get the same flight tools as admins — any restriction would be backend-enforced), plus `AccountSettingsView` (change password + danger-zone deactivate/delete).

Retail — Admin (`retail-admin-dashboard.tsx`)

Tabs: **Stores · Products · Inventory · Staff · Analytics · Profile**. This is the retail counterpart of the aviation admin dashboard, built to mirror it visually. - **`RetailStoresView`** — create/edit/delete stores, assign a **responsible employee** from a dropdown of staff, and a detail modal that shows that store's stock. Loads stores + employees + products + inventory together. - **`RetailProductsView`** — create products (name, SKU, price, description) with **variant options** (a Color/Size/custom builder — comma-separated values), list the catalog, and show each product's options as badges. - **`RetailInventoryView`** — the admin stock manager. "Assign stock" (product × store × quantity), edit a quantity, delete a record. Low-stock rows get an amber badge (threshold = 10). - **Staff** reuses `AdminEmployeesView` (same component as aviation — these are the employees who become store managers). - **`RetailAnalyticsView`** — computed **client-side** from the real product/inventory/store data: active stores, total units, inventory value (Σ quantity × price), low-stock alerts, and a per-store performance table. - `ProfileView`.

Retail — Employee (`retail-employee-dashboard.tsx`)

Tabs: **Inventory · Analytics · Settings**. - **`RetailEmployeeInventoryView`** — scoped to the store(s) the employee manages (matched via `store.manager_id === getUserId()`; falls back to all inventory with a banner if they're unassigned). It shows a **Suggested Actions** panel (client-side heuristics that flag out-

of-stock in red and low-stock in amber) and lets the employee update quantities with +/- steppers. - `RetailAnalyticsView` **scoped to the store(s) they manage** (via the `managerId` prop = their user id) — an employee sees only their own store's stats, never another store's. The admin's analytics stays brand-wide. `AccountSettingsView` rounds out the tab set.

How the retail pieces connect

Admin creates a **staff member** → assigns them as a store's **responsible employee** → that employee logs in and sees *their* store's inventory and suggested actions. The link is the new `manager_id` on the store plus the `userId` the frontend persists at login.

---

7. The "other" dashboard system (mostly dead code)

There are actually **two** dashboard systems in the repo:

1. **The role-based dashboards** (§6) — real, API-driven, what users see.
2. **The config-driven `Dashboard`** (`dashboard.tsx` + `dashboard-chrome.tsx`, `dashboard-content.tsx`, `dashboard-account.tsx`) — fed by the mock data in `lib/industries.ts`. It includes a settings view, a **fully mock** billing view, stat cards, a directory table, and an ops table.

The second system only renders when `userRole` is `null`, which never happens after a real login — so it's effectively **unreachable demo scaffolding**. It's worth knowing it exists (it's a lot of files) but it isn't part of the live product. This is a natural cleanup candidate; see known-issues.md.

---

8. The styling system (`app/globals.css`)

The visual language is a **design-token system** built on Tailwind v4:

- A set of CSS variables defines the palette — `--background`, `--foreground`, `--card`, `--primary`, `--accent`, `--muted`, `--border`, `--input`, `--destructive`, plus chart and sidebar tokens. Tailwind's `@theme inline` maps each to a color utility (so `bg-card`, `text-muted-foreground`, `border-border`, etc. all resolve to these variables).
- **Light mode** is a warm "editorial paper" theme with an evergreen accent; **dark mode** (`.dark`, toggled by `next-themes`) is a neutral near-black gray palette. (The two aren't perfectly matched — the evergreen accent largely disappears in dark mode; a known cosmetic issue.)

Because every component uses the same tokens, the whole app shares a consistent look. The recurring UI patterns you'll see everywhere:

- **Cards:** `rounded-xl border border-border bg-card/40`.
- **Tables:** an uppercase, muted `<thead>` with `border-b`, and rows that highlight on hover (`hover:bg-muted/30`); many rows are clickable to open a detail modal.
- **Modals:** a full-screen `fixed inset-0 bg-foreground/40` overlay with a centered card that stops click propagation.
- **Status badges:** pill shapes (`rounded-full border px-2.5 py-1 text-xs`) tinted emerald / amber / rose for ok / warning / danger.
- **Forms:** `rounded-lg border border-input bg-background` inputs with an accent focus ring.

The retail components were written to reuse these exact patterns, so the retail dashboards are visually indistinguishable in style from the aviation ones — which was the goal.

---

9. Quick reference — where things live

You want to change…	Look in
Which dashboard renders for a role/industry	`app/page.tsx`
Login/signup UI & flow	`components/auth-screen.tsx`
Any backend call	`lib/api.ts`
Retail store management	`components/retail-stores.tsx`
Retail product catalog	`components/retail-products.tsx`
Retail stock assignment (admin)	`components/retail-inventory.tsx`
Retail stock editing + suggestions (employee)	`components/retail-employee-inventory.tsx`
Retail analytics math	`components/retail-analytics.tsx`
Colors / theme tokens	`app/globals.css`

MCP Server Documentation

The MCP server is what makes UniFlow "AI-native." It lets an AI assistant — ChatGPT, Claude, or any Model Context Protocol client — **take real actions in the system** by calling functions, rather than just

talking about them.

File: `backend/app/mcp/server.py`.

---

1. What MCP is, and why it's here

MCP (Model Context Protocol) is an open standard for exposing "tools" (functions) to an AI model. You describe a set of typed functions; the AI decides when to call them, with what arguments, and reads back the results. It's the mechanism behind "AI that can actually do things" instead of just generating text.

In UniFlow, the MCP server turns the backend into something an end **customer** can operate through conversation. Instead of navigating a website, a customer can say:

- *"Find me a flight from Algiers to Paris."*
- *"Book seat in business class for John, john@email.com."*
- *"Is the wireless headphone in stock near downtown?"*
- *"Which of your stores has the running shoes?"*

…and the AI calls the matching tool, which calls the backend, which returns real data.

The key architectural point

The MCP server **does not touch the database**. It's a thin translation layer:

```
AI assistant ──calls tool──> MCP server ──HTTP request──> Backend API ──> PostgreSQL
             <──result─────              <──JSON─────────
```

Every tool is a small Python function that uses the `requests` library to hit a backend HTTP endpoint (`BASE_URL = http://127.0.0.1:8000`) and returns the JSON. This is why the **backend must be running before the MCP server**, and why the MCP tools can only reach the backend's **public / guest** endpoints — the AI has no login, so it can only call endpoints that don't require one.

Built with FastMCP

The server uses **FastMCP**. Each tool is just a function decorated with `@mcp.tool()`; FastMCP handles the protocol, argument typing, and schema generation automatically. It's served over **streamable-HTTP on port 8001**:

```
mcp = FastMCP("aviation business MCP")
BASE_URL = "http://127.0.0.1:8000"
```

```
@mcp.tool()
def flight_search(origin=None, destination=None, company_name=None):
    ...
    return requests.get(f"{BASE_URL}/flights/public/search", params=params).json()

if __name__ == "__main__":
    mcp.run(transport="streamable-http", port=8001)
```

---

2. The tools

There are **fourteen** tools, in two groups: the original **aviation/booking** tools and the **retail** tools added later.

2.1 Utility

Tool	Calls	Purpose
`health_check()`	—	Returns `{status: ok}`; a simple "is the MCP server alive?" probe.

2.2 Aviation & booking tools

These wrap the flight and guest-booking endpoints — a complete customer journey from search to paid ticket.

Tool	Backend endpoint	What a customer uses it for
`flight_search(origin?, destination?)`	`GET /flights/public/search`	"Find flights from X to Y."
`check_seat_availability(flight_id)`	`GET /flights/{id}/available`	"How many seats are left, by class?"
`check_flight_status(flight_id)`	`GET /flights/{id}/status`	"Is flight 123 on time?"
`reserve_ticket(flight_id,`	`POST`	"Book me a ticket" — creates the

Tool	Backend endpoint	What a customer uses it for
`flight_class_id, passenger_name, passenger_email)`	`/bookings/guest`	reservation and triggers the payment-link email.
`get_booking_details(booking_id, passenger_email)`	`GET /bookings/guest/{id}`	"Show me my booking."
`update_booking_email(booking_id, old_email, new_email)`	`PUT /bookings/guest/{id}/email`	"Change the email on my booking."
`cancel_booking(booking_id, passenger_email)`	`PUT /bookings/guest/{id}/cancel`	"Cancel my booking."
`pay_booking(booking_id, passenger_email)`	`PUT /bookings/guest/{id}/pay`	"Pay for my booking" (simulated payment).

2.3 Retail tools

These were added to give retail customers the same conversational experience. They're all **read-only** — a customer browses and checks availability but doesn't modify a brand's data.

Tool	Backend endpoint	What a customer uses it for
`search_products(query)`	`GET /products/public/search`	"Do you sell wireless headphones?" (matches name **or** SKU).
`get_product_details(product_id)`	`GET /products/public/{id}`	"Tell me the price and details of this product."
`check_product_availability(product_name, location?)`	`GET /inventory/public/availability`	"Is product X in stock near me?" — the single most useful retail tool.
`list_stores(location?)`	`GET /stores/public/search`	"Where are your shops?"
`find_stores_with_product(product_name)`	`GET /inventory/public/availability`	"Which store has product X?" (same endpoint, no location filter).

None of the tools take a `company_name` anymore — the server is **bound to one brand by its API key**

(below), so every tool is automatically scoped to that company.

Two design details worth noting:

- **`check_product_availability` and `find_stores_with_product` share one endpoint.** That endpoint joins inventory → product → store and returns a **flat, human-readable shape** (product name, store name, location, quantity) rather than raw ids — because an AI works far better with "3 units of Running Shoes at Downtown Store" than with `{product_id: 7, store_id: 2, quantity: 3}`.
- **Per-brand isolation via API key.** The server reads a `COMPANY_API_KEY` from its environment and sends it as an `X-API-Key` header on every tool call. The backend resolves it to a company and scopes all results to that brand — so one deployment of the MCP server represents exactly one company and can never read another's data. Set the key from the admin dashboard (`GET /companies/me/api-key`). Without a key, the tools fall back to searching across all brands (fine for a shared demo).

2.4 The natural two-step pattern

The retail tools are designed to chain. A customer speaks in **names**, but `get_product_details` needs a numeric **id**. So the AI naturally does:

1. `search_products("headphones")` → reads back the product and its `id`.
2. `get_product_details(id)` → price and description.

The AI handles this chaining on its own — no special glue is needed.

---

3. Why these tools are read-only for retail (and writeable for booking)

There's a deliberate asymmetry:

- **Booking tools can write** (reserve, pay, cancel) because those are **customer-owned actions** on the customer's *own* booking, done entirely through guest endpoints that verify the passenger's email.
- **Retail tools are read-only** because creating stores, products, or inventory are **admin/back-office actions**, not something a shopping customer should do. Those operations stay in the authenticated web dashboard, behind a login. (A store owner adding a shop from a map, for instance, would need an authenticated widget — that's out of scope for the public MCP layer.)

This split keeps the AI surface safe: an AI client with no credentials can *discover and transact on its own bookings*, but can never mutate a brand's catalog or another customer's data.

---

4. Running the MCP server

```
cd backend
source venv/bin/activate
COMPANY_API_KEY=<the brand's key> python app/mcp/server.py
```

`COMPANY_API_KEY` binds this server to one brand (omit it only for a shared, all-brands demo). This starts the MCP server on **port 8001** over streamable-HTTP. Because every tool calls `http://127.0.0.1:8000`, **start the backend first** (see README.md §6.1). A quick way to confirm both are healthy: call the `health_check` tool, then `flight_search` — if the backend is up and has data, you'll get results back.

---

5. Connecting an AI client

The MCP server speaks the standard protocol over HTTP on `localhost:8001`, so any MCP-capable client can connect by pointing at that URL. In practice you register the server in the client's MCP configuration (for example, a `mcpServers` entry) with the streamable-HTTP endpoint; the client then discovers all fourteen tools automatically from FastMCP's generated schema and can start calling them.

> Note on ChatGPT specifically: exposing these tools *inside the ChatGPT app* as an interactive experience (custom UI, maps, etc.) is a larger effort that uses OpenAI's Apps SDK on top of MCP. The current server provides the tool layer — the actions the AI can take — which is the foundation any such integration builds on.

---

6. Where this is heading

The `mcp_tool_permissions`, `rate_limits`, `api_integrations`, and `audit_logs` tables in the database (see database.md §6) are scaffolding for the *governed* version of this: per-company control over which tools are enabled, rate limiting so an AI can't hammer the backend, the ability to register a company's own external APIs as tools, and an audit trail of everything the AI did. None of that is wired up yet, but the MCP server as it stands is the working core those features would wrap around.

API Reference

A compact lookup table of every backend HTTP endpoint. For the narrative explanation of what each

does and why, see backend.md.

Auth column legend: - PUBLIC **Public** — no token required (customer/guest/MCP facing) - AUTH **Auth** — any authenticated user (Bearer token); tenant-scoped to the caller's company - ADMIN **Admin** — authenticated **and** `role == "admin"`

Base URL: `http://localhost:8000`. Interactive docs: `http://localhost:8000/docs`.

Auth — `/auth`

Method	Path	Auth	Purpose
POST	`/auth/register-company`	PUBLIC	Create company + admin user; sends verification email
POST	`/auth/Login`	PUBLIC	Log in (blocks unverified users); returns JWT
GET	`/auth/me`	AUTH	Current user profile incl. `company_name` + `industry`
GET	`/auth/verify-email?token=`	PUBLIC	Confirm email address
POST	`/auth/resend-verification`	PUBLIC	Re-send verification email
POST	`/auth/forget-password`	PUBLIC	Request a password-reset email (1h token)
POST	`/auth/reset-password`	PUBLIC	Reset password using a token
PUT	`/auth/change-password`	AUTH	Change own password (needs current password)
POST	`/auth/register-employee`	ADMIN	Create a staff member in your company

Admin — `/admin` (all ADMIN)

Method	Path	Purpose
GET	`/admin/employees`	List all employees in the company

Method	Path	Purpose
PUT	`/admin/employees/{id}`	Update name / role / is_active
POST	`/admin/employees/{id}/disable`	Disable an employee
POST	`/admin/employees/{id}/password`	Admin reset of an employee's password
PUT	`/admin/flights/{id}`	Update any flight field (incl. status)
DELETE	`/admin/flights/{id}`	Delete a flight (blocked if reserved bookings exist)
GET	`/admin/bookings/{id}`	Booking detail enriched with flight + class

Account — `/account`

Method	Path	Auth	Purpose
POST	`/account/deactivate`	AUTH	Deactivate own account (blocked if sole admin)
DELETE	`/account/delete`	AUTH	Delete own account; sole-admin triggers company wipe

Companies — `/companies`

Method	Path	Auth	Purpose
GET	`/companies/me`	AUTH	Get the caller's company
PUT	`/companies/me`	ADMIN	Update company name / industry / email
GET	`/companies/me/api-key`	ADMIN	Get the brand's MCP API key (for `COMPANY_API_KEY`)
POST	`/companies/me/api-key/rotate`	ADMIN	Rotate the brand's MCP API key

> **`X-API-Key` header:** the public discovery endpoints below (flights/products/stores/inventory search) accept an optional `X-API-Key`. When present, results are scoped to that brand only and

> `company_name` is ignored; an invalid key returns `401`. This is how the MCP server isolates one company's data.

Flights — `/flights` *(aviation)*

Method	Path	Auth	Purpose
POST	`/flights/`	AUTH	Create a flight with cabin classes
GET	`/flights/`	AUTH	List company flights (origin/destination filters)
GET	`/flights/{id}`	AUTH	One flight
GET	`/flights/public/search`	PUBLIC	Search flights (origin/destination/company_name)
GET	`/flights/{id}/available`	PUBLIC	Seats available by class
GET	`/flights/{id}/status`	PUBLIC	Flight number + status

Bookings — `/bookings` *(aviation)*

Method	Path	Auth	Purpose
POST	`/bookings/`	AUTH	Staff-side booking
GET	`/bookings/`	AUTH	List bookings *(currently returns a single row — see known-issues)*
PUT	`/bookings/{id}/cancel`	AUTH	Cancel a booking (restores a seat)
POST	`/bookings/guest`	PUBLIC	Guest booking + payment-link email
GET	`/bookings/guest/{id}?passenger_email=`	PUBLIC	Look up a guest booking
PUT	`/bookings/guest/{id}/pay?passenger_email=`	PUBLIC	Simulated payment

Method	Path	Auth	Purpose
PUT	`/bookings/guest/{id}/cancel?passenger_email=`	PUBLIC	Guest cancel
PUT	`/bookings/guest/{id}/email?old_email=&new_email=`	PUBLIC	Change passenger email, re-send payment email

Products — `/products` *(retail)*

Method	Path	Auth	Purpose
POST	`/products/`	AUTH	Create a product (unique SKU per company)
GET	`/products/`	AUTH	List company products
GET	`/products/public/search?query=&company_name=`	PUBLIC	Search by name or SKU
GET	`/products/public/{id}`	PUBLIC	Single product details

Stores — `/stores` *(retail)*

Method	Path	Auth	Purpose
POST	`/stores/`	AUTH	Create a store (name, location, optional manager)
GET	`/stores/`	AUTH	List stores (with manager name)
PUT	`/stores/{id}`	AUTH	Update store (name/location/manager)
DELETE	`/stores/{id}`	AUTH	Delete store (clears its inventory first)
GET	`/stores/public/search?location=&company_name=`	PUBLIC	Public store search

Inventory — `/inventory` *(retail)*

Method	Path	Auth	Purpose
POST	`/inventory/`	AUTH	Assign stock (adds to existing product×store, else creates)
GET	`/inventory/`	AUTH	List company inventory
PUT	`/inventory/{id}`	AUTH	Overwrite a record's quantity
DELETE	`/inventory/{id}`	AUTH	Remove a stock record
GET	`/inventory/public/availability?product_name=&location=&company_name=`	PUBLIC	In-stock rows joined with product + store names

---

MCP tools (port 8001)

Not HTTP endpoints per se — these are the AI-callable tools, each of which calls one of the PUBLIC endpoints above. Full detail in mcp.md.

Tool	Backend endpoint
`health_check`	—
`flight_search`	`GET /flights/public/search`
`check_seat_availability`	`GET /flights/{id}/available`
`check_flight_status`	`GET /flights/{id}/status`
`reserve_ticket`	`POST /bookings/guest`
`get_booking_details`	`GET /bookings/guest/{id}`
`update_booking_email`	`PUT /bookings/guest/{id}/email`
`cancel_booking`	`PUT /bookings/guest/{id}/cancel`
`pay_booking`	`PUT /bookings/guest/{id}/pay`
`search_products`	`GET /products/public/search`
`get_product_details`	`GET /products/public/{id}`

Tool	Backend endpoint
`check_product_availability`	`GET /inventory/public/availability`
`list_stores`	`GET /stores/public/search`
`find_stores_with_product`	`GET /inventory/public/availability`

Known Issues, Limitations & Future Work

This document is deliberately honest. For a final-year project it's valuable to show that you *know* where the sharp edges are — a "Limitations & Future Work" section is often expected in the report. Nothing here blocks the app from running; these are correctness, security, and cleanliness items ranked roughly by importance.

---

1. Bugs (functional correctness)

#	Where	Issue	Effect
1	`booking.py` (guest create + email change)	`email_status = "sent",` has a **trailing comma**, making the value a tuple `("sent",)` instead of the string `"sent"`.	The `email_status` column stores a tuple-looking value; the `BookingResponse` schema expects a string.
2	`booking.py` `GET /bookings/`	Uses `.first()` with a **singular** `response_model`, despite being intended as "list all bookings."	Staff booking list returns only one booking (or null) instead of all.
3	`models/Store.py`	Unique constraint uses `__table_arg__` (typo) instead of `__table_args__`, and isn't a tuple.	The intended (company_id, name) uniqueness **is never created** — duplicate store names are allowed.
4	`deps.py` / `auth.py` login	Neither login nor `get_current_user` checks **`is_active`**.	Disabling/deactivating a user does **not** actually block them from logging in or using the API — the feature is

#	Where	Issue	Effect
			cosmetic right now.
5	`account.py` `DELETE /account/delete`	The sole-admin "company wipe" deletes bookings/flights/users/company but **not** `products`, `stores`, `inventory`.	Deleting a company that has retail data will hit a foreign-key violation.
6	`inventory.py` `DELETE /inventory/{id}`	Returns a Python **set literal** `{"...deleted..."}` instead of a dict.	Slightly malformed response body (still 200, but not a proper JSON object).
7	`auth.py` `resend-verification`	Guard `if not user or not user.is_verified` is **inverted** — it only proceeds for already-verified users.	Resending verification to a genuinely unverified user may not behave as intended.
8	`email.py` `password_resert_email`	The reset email links to `/verify-emai` (the verification page), not a reset page.	Users clicking a reset link land on the wrong screen.

---

2. Security items

#	Item	Detail	Recommendation
1	Secrets in `.env`	`SECRET_KEY` is a weak, human-readable string; Gmail credentials are committed.	Rotate to a strong random `SECRET_KEY`; ensure `.env` is git-ignored; use an app-specific mail credential in a secret manager.
2	Long token lifetime	JWT expiry is ~**14.6 days** (`30 * 700` minutes).	Shorten to hours; add a refresh-token flow if long sessions are needed.
3	Unvalidated employee role	`register-employee` takes `role` straight from the body with no whitelist (only `admin/employee` is enforced on *update*).	Whitelist `role` on creation too.
4	No rate limiting	Public/guest and MCP-facing endpoints have no throttling.	The `rate_limits` table is scaffolded

#	Item	Detail	Recommendation
			for exactly this — wire it up.
5	Booking seat race	`available_seats` is read-then-decremented without a lock.	Concurrent bookings could oversell; use a row lock or atomic decrement.
6	MCP API keys stored in plaintext	`companies.api_key` is stored as-is; a DB leak exposes every brand's key.	Store a hash (e.g. `sha256`) and compare on lookup, like passwords.
7	Per-brand scoping is opt-in	The public discovery endpoints still allow no-key access (falls back to all brands).	For strict production, require the key (drop the fallback), or keep it deliberately as a public marketplace.

---

3. Correctness / semantics smells

- **HTTP status codes:** several "already exists / already done" cases return **`404`** where `409`/`400` would be correct (`register-company` "email already used", booking already-cancelled/already-paid). Functional, but semantically off.
- **`update_flight`** writes ISO **strings** into `DateTime` columns; **`delete_flight`** reads `flight.flight_number` *after* deleting and committing the row.
- **Route casing:** `/auth/Login` (capital L) is inconsistent with every other lowercase path.
- **Duplicated constant:** `ACCESS_TOKEN_EXPIRE_MINUTES` is defined identically in both `security.py` and `deps.py`.

---

4. Dead / duplicate code

- **Two `User` models:** `models/user.py` (`User`, used) and `models/users.py` (`Users`, unused legacy) both map to the `users` table. `Users` should be deleted.
- **`role` default casing:** the `User.role` column default is `"Admin"` (capitalized) while all logic uses lowercase — harmless today because role is always set explicitly, but a trap.
- **The config-driven dashboard system** (`dashboard.tsx`, `dashboard-chrome.tsx`, `dashboard-content.tsx`, `dashboard-account.tsx`, fed by the mock data in `lib/industries.ts`) is only reachable when `userRole` is `null`, which never happens after login. It's effectively **dead demo code** — a cleanup candidate.

- **Mock data in the frontend:** the billing view, the settings email/website fields, and everything in `lib/industries.ts` (stats, directories, ops tables) are hardcoded placeholders, not live data.
- **Misspelled route folder** `app/verify-emai/` (missing the "l"); the backend links to it to match, so fixing one means fixing both.
- **Undefined CSS utilities** `.text-gradient` and `.glow-violet` are referenced in markup but not defined in `globals.css` (they no-op).
- **Five unused backend models** (`api_integrations`, `endpoints`, `mcp_tool_permissions`, `rate_limits`, `audit_logs`) — scaffolding, not bugs, but currently inert. **No audit logging exists yet.**

---

5. Retail feature — current limitations

The retail half is newer. Things intentionally left simple:

- **Analytics is computed client-side** in the browser from `/products` + `/inventory`, not by a backend endpoint. Fine for a single company's small dataset; promote to a server-side aggregation endpoint if data grows.
- **Products have no edit/delete** endpoints or UI (only create + list). Stores and inventory have full CRUD; products don't yet.
- **The employee→store link is single-manager.** A store has one `manager_id`; there's no many-to-many "staff assigned to a store." An employee sees the store(s) where they are *the* manager.
- **Low-stock threshold is a hardcoded** `10` in the frontend, not configurable per company/product.
- **No map / geolocation** for stores — `location` is a free-text string. A "pick your store on a map" feature would need lat/lng columns and (for in-chat map picking) OpenAI's Apps SDK.

---

6. Suggested future-work roadmap

A natural ordering if the project continues:

1. **Fix the functional bugs** in §1 (especially #4 `is_active` enforcement, #1 the tuple bug, #2 the booking list, #5 the delete cascade).
2. **Harden security** (§2): strong secret, shorter tokens, role whitelist.
3. **Real payments** — the current `pay_booking` is a simulation; the code comments already anticipate Stripe / Chargily / SATIM / CIB webhooks.
4. **Backend analytics endpoints** for retail, replacing the client-side computation.

5. **Complete product CRUD** (edit/delete).
6. **Activate the governance layer** — use the scaffolded tables to add per-company MCP tool permissions, rate limiting, and audit logging.
7. **Frontend cleanup** — delete the dead config-driven dashboard system and the unused `Users` model; fix the misspelled route.
8. **A General industry** (if desired) — add the enum value, build a `GeneralDashboard`, route to it, and re-enable the dropdown option.

---

7. What was fixed during the retail build (for context)

So the history is clear, these were found and fixed while adding the retail side:

- **`main.py` mounted only 3 of 9 routers** — the entire retail API plus `/admin` and `/account` returned `404`. Now all nine are mounted (and the duplicate `auth` removed).
- **`/auth/me` didn't return `industry`/`company_name`** — every account resolved to the aviation dashboard on login. Fixed by joining the company row.
- **`create_store` assigned the whole `User` object to `company_id`** instead of `current_user.company_id`. Fixed.
- **Stores had no `manager_id`** — added the column, a migration, and the create/update/validation logic for the "responsible employee" requirement.
- **"General" was offered in signup** but unsupported by the backend enum — removed from the dropdown.